Chapter 1: Introduction

C is (as K&R admit) a relatively small language, but one which (to its admirers, anyway) wears well. C's small, unambitious feature set is a real advantage: there's less to learn; there isn't excess baggage in the way when you don't need it. It can also be a disadvantage: since it doesn't do everything for you, there's a lot you have to do yourself. (Actually, this is viewed by many as an additional advantage: anything the language doesn't do for you, it doesn't dictate to you, either, so you're free to do that something however you want.)

C is sometimes referred to as a ``high-level assembly language.'' Some people think that's an insult, but it's actually a deliberate and significant aspect of the language. If you have programmed in assembly language, you'll probably find C very natural and comfortable (although if you continue to focus too heavily on machine-level details, you'll probably end up with unnecessarily nonportable programs). If you haven't programmed in assembly language, you may be frustrated by C's lack of certain higher-level features. In either case, you should understand why C was designed this way: so that seemingly-simple constructions expressed in C would not expand to arbitrarily expensive (in time or space) machine language constructions when compiled. If you write a C program simply and succinctly, it is likely to result in a succinct, efficient machine language executable. If you find that the executable program resulting from a C program is not efficient, it's probably because of something silly you did, not because of something the compiler did behind your back which you have no control over. In any case, there's no point in complaining about C's low-level flavor: C is what it is.

A programming language is a tool, and no tool can perform every task unaided. If you're building a house, and I'm teaching you how to use a hammer, and you ask how to assemble rafters and trusses into gables, that's a legitimate question, but the answer has fallen out of the realm of ``How do I use a hammer?'' and into ``How do I build a house?''. In the same way, we'll see that C does not have built-in features to perform every function that we might ever need to do while programming.

As mentioned above, C imposes relatively few built-in ways of doing things on the programmer. Some common tasks, such as manipulating strings, allocating memory, and doing input/output (I/O), are performed by calling on library functions. Other tasks which you might want to do, such as creating or listing directories, or interacting with a mouse, or displaying windows or other user-interface elements, or doing color graphics, are not defined by the C language at all. You can do these things from a C program, of course, but you will be calling on services which are peculiar to your programming environment (compiler, processor, and operating system) and which are not defined by the C standard. Since this course is about portable C programming, it will also be steering clear of facilities not provided in all C environments.

Another aspect of C that's worth mentioning here is that it is, to put it bluntly, a bit dangerous. C does not, in general, try hard to protect a programmer from mistakes. If you write a piece of code which will (through some oversight of yours) do something wildly different from what you intended it to do, up to and including deleting your data or trashing your disk, and if it is possible for the compiler to compile it, it generally will. You won't get warnings of the form ``Do you really mean to...?'' or ``Are you sure you really want to...?''. C is often compared to a sharp knife: it can do a surgically precise job on some exacting task you have in mind, but it can also do a surgically precise job of cutting off your finger. It's up to you to use it carefully.

This aspect of C is very widely criticized; it is also used (justifiably) to argue that C is not a good teaching language. C aficionados love this aspect of C because it means that C does not try to protect them from themselves: when they know what they're doing, even if it's risky or obscure, they can do it. Students of C hate this aspect of C because it often seems as if the language is some kind of a conspiracy specifically designed to lead them into booby traps and ``gotcha!''s.

This is another aspect of the language which it's fairly pointless to complain about. If you take care and pay attention, you can avoid many of the pitfalls. These notes will point out many of the obvious (and not so obvious) trouble spots.

Read sequentially: prev next up top